Plataforma EAD em Microservicos

Documentacao tecnica, funcional, arquitetura, modelo de dados e especificacoes. Estado do projeto em 30/10/2026.

Area	Resumo
Objetivo	Operar cursos EAD com matricula por CPF, aulas em cards, liberacao semanal, progresso e pagamento por aula.
Arquitetura	Frontend React, Nginx gateway, sete microservicos FastAPI e bancos PostgreSQL isolados.
Estado	Fluxo principal implementado; atividade ENADE e debito semanal ainda pendentes.

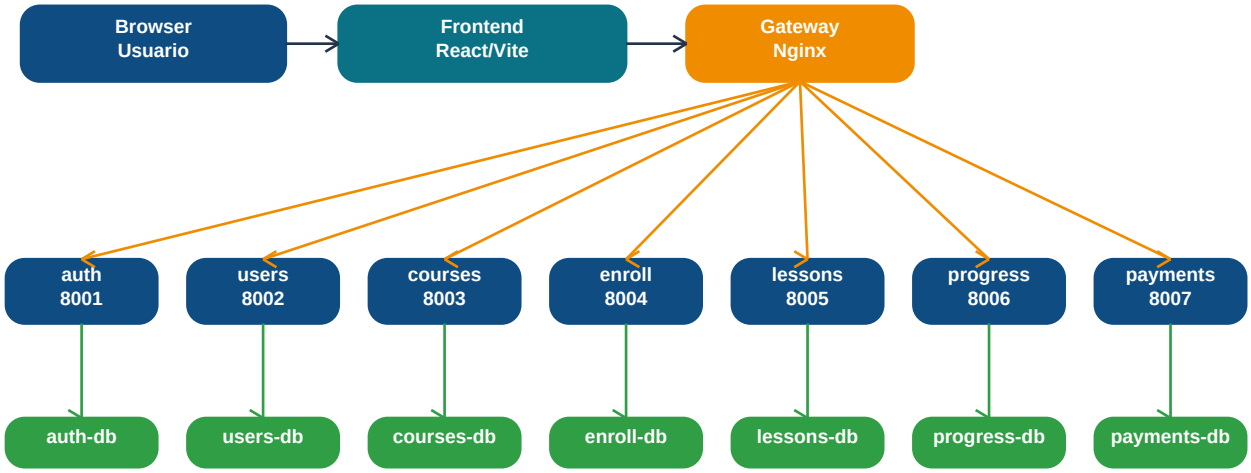
Requisitos funcionais

ID	Requisito	Status
RF-01	Registrar e autenticar usuarios com roles admin e aluno.	Implementado
RF-02	Manter perfil do aluno, CPF unico e dados de contato.	Implementado
RF-03	Consultar cursos e permitir gestao de cursos e turmas por admin.	Implementado
RF-04	Matricular por CPF na janela permitida, respeitando capacidade, grupos e prazos.	Implementado
RF-05	Criar aulas multimidia em sequencia, ate 40 por curso.	Implementado
RF-06	Liberar aulas semanalmente e exigir conclusao sequencial.	Implementado
RF-07	Registrar conclusao e exibir percentual e proxima aula.	Implementado
RF-08	Cobrar aula paga antes da conclusao e evitar duplicidade.	Parcial: sem provedor real
RF-09	Exibir progresso do aluno e visao administrativa da turma.	Implementado
RF-10	Aplicar prazo de entrega de 180 dias e acesso por 360 dias.	Implementado
RF-11	Disponibilizar avaliacao de 30 questoes no padrao ENADE.	Pendente
RF-12	Efetuar debito financeiro no fim da semana.	Pendente

Requisitos nao funcionais

ID	Requisito e estado observado
RNF-01 Seguranca	JWT, roles e hash de senha implementados. Para producao: substituir segredos padrao, usar HTTPS e revisar armazenamento do token.
RNF-02 Privacidade	Numero completo do cartao e CVV nao sao persistidos. Definir politica de privacidade, retencao e conformidade legal.
RNF-03 Persistencia	Bancos e arquivos usam volumes Docker; backup e restauracao nao estao documentados.
RNF-04 Implantacao	Docker Compose configura frontend, gateway, servicos, bancos e volumes.
RNF-05 Interoperabilidade	APIs REST/JSON com documentacao OpenAPI/Swagger por servico.
RNF-06 Manutenibilidade	Sete servicos e bancos isolados; integridade entre servicos e logica.
RNF-07 Disponibilidade	Health checks e reinicio de containers; sem SLA ou alta disponibilidade definidos.
RNF-08 Observabilidade	Logs basicos e health checks; sem metricas, tracing ou logs centralizados.
RNF-09 Desempenho	Limite de 200 alunos por turma; sem metas de latencia ou testes de carga documentados.

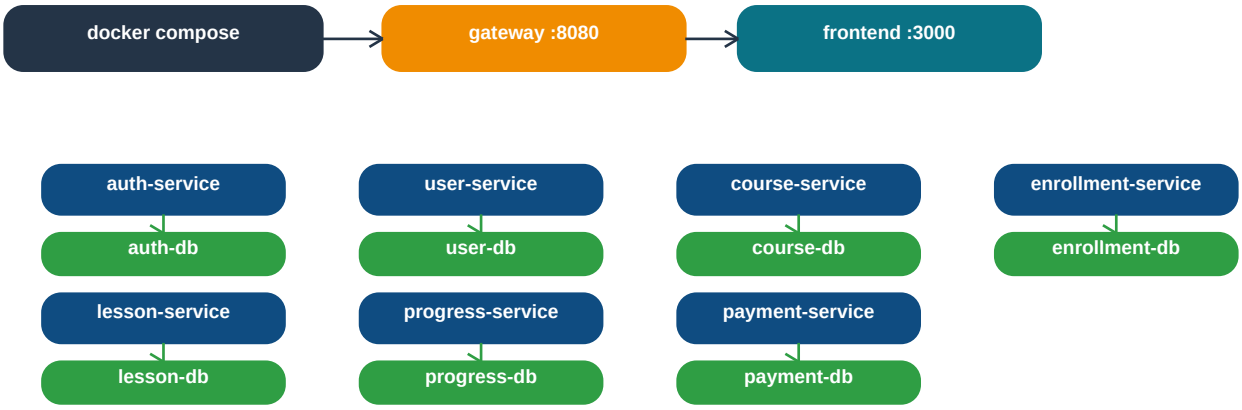
1. Arquitetura



Dependencias internas: enrollment -> users/courses; progress -> enrollments/lessons/payments/users; payments -> courses/lessons/enrollments.

O frontend acessa apenas o gateway. O gateway roteia as familias de rotas para os microservicos. Cada servico possui seu banco PostgreSQL proprio, preservando isolamento operacional.

2. Stack e implantacao



Camada	Tecnologia
Frontend	React 18, Vite, Tailwind CSS
Gateway	Nginx 1.27 Alpine
Backend	FastAPI, Pydantic, SQLAlchemy
Auth	JWT HS256, python-jose, passlib/bcrypt
Banco	PostgreSQL 16, um banco por servico
Orquestracao	Docker Compose

3. Servicos e contratos

auth-service

Registro, login, JWT e roles.

Metodo	Rota	Descricao
GET	/health	Saude do servico
POST	/auth/register	Cria usuario aluno ou admin
POST	/auth/login	Autentica e retorna JWT
GET	/auth/me	Retorna usuario autenticado

- Roles permitidas: admin e aluno.

- Admin exige ADMIN_REGISTRATION_CODE.

- JWT contem sub, name, email, role e exp.

Tabela principal: users.

user-service

Perfil do aluno por CPF.

Metodo	Rota	Descricao
GET	/health	Saude do servico
GET	/users	Lista alunos; filtros cpf e auth_user_id
POST	/users	Cria perfil
PUT	/users/{user_id}	Atualiza perfil
DELETE	/users/{user_id}	Remove perfil

- CPF unico.

- Usuario comum so acessa o proprio perfil.

- Campos incluem WhatsApp, Telegram, cidade, estado e escolaridade.

Tabela principal: students.

course-service

Catalogo, turmas e janela de matricula.

Metodo	Rota	Descricao
GET	/health	Saude do servico
GET	/courses	Lista cursos
GET	/courses/{course_id}	Detalha curso
POST	/courses	Cria curso; admin
PUT	/courses/{course_id}	Atualiza curso; admin

- Capacidade maxima de 200 alunos.

- Janela calculada: 5 a 1 semanas antes do inicio.

- Curso pago deriva de price > 0 quando nao informado.

Tabela principal: courses.

enrollment-service

Matricula por CPF.

Metodo	Rota	Descricao
GET	/health	Saude do servico
POST	/enrollments	Cria matricula
GET	/enrollments/user/{user_id}	Lista matriculas do usuario
GET	/enrollments/course/{course_id}	Lista matriculas da turma; admin

- Busca aluno por CPF.
- Matricula permitida apenas na janela operacional.
- Grupos automaticos de ate 5 alunos.
- Final delivery em 180 dias e acesso em 360 dias.

Tabela principal: enrollments.

lesson-service

Curriculo, aulas e cards multimidia.

Metodo	Rota	Descricao
GET	/health	Saude do servico
GET	/lessons/course/{course_id}	Lista aulas do curso
GET	/lessons/{lesson_id}	Detalha aula
POST	/lessons	Cria aula; admin

- Maximo de 40 aulas por curso.
- Video e texto duram 30 min; atividade dura 60 min.
- Cards: texto, imagem, video, PDF, link e embed.
- release_week segue order_index.

Tabela principal: lessons.

progress-service

Progresso, sequencia e leitura administrativa da turma.

Metodo	Rota	Descricao
GET	/health	Saude do servico
POST	/progress	Marca aula concluida
GET	/progress/{user_id}	Progresso por curso
GET	/progress/course/{course_id}/students	Progresso por aluno; admin

- Valida matricula, acesso, liberacao semanal e ordem.
- Em aula paga chama payment-service antes de registrar progresso.
- Admin enxerga percentual, proxima aula e dados do aluno.

Tabela principal: progress_entries.

payment-service

Pagamento por aula no cartao.

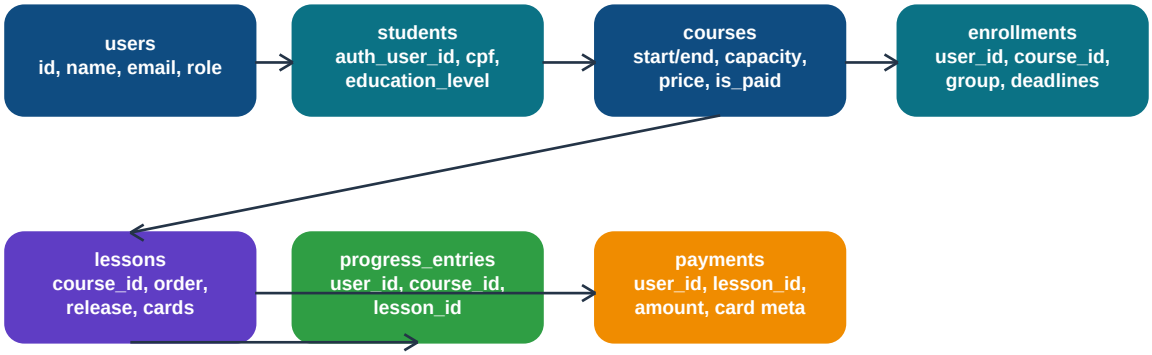
Metodo	Rota	Descricao
GET	/health	Saude do servico

Metodo	Rota	Descricao
POST	/payments	Registra pagamento de aula
GET	/payments/{user_id}	Lista pagamentos do usuario

- Nao persiste numero completo do cartao nem CVV.
- Impede duplicidade por usuario/aula paga.
- Valor precisa ser exatamente igual ao preco da aula.
- Valida matricula, liberacao e acesso.

Tabela principal: payments.

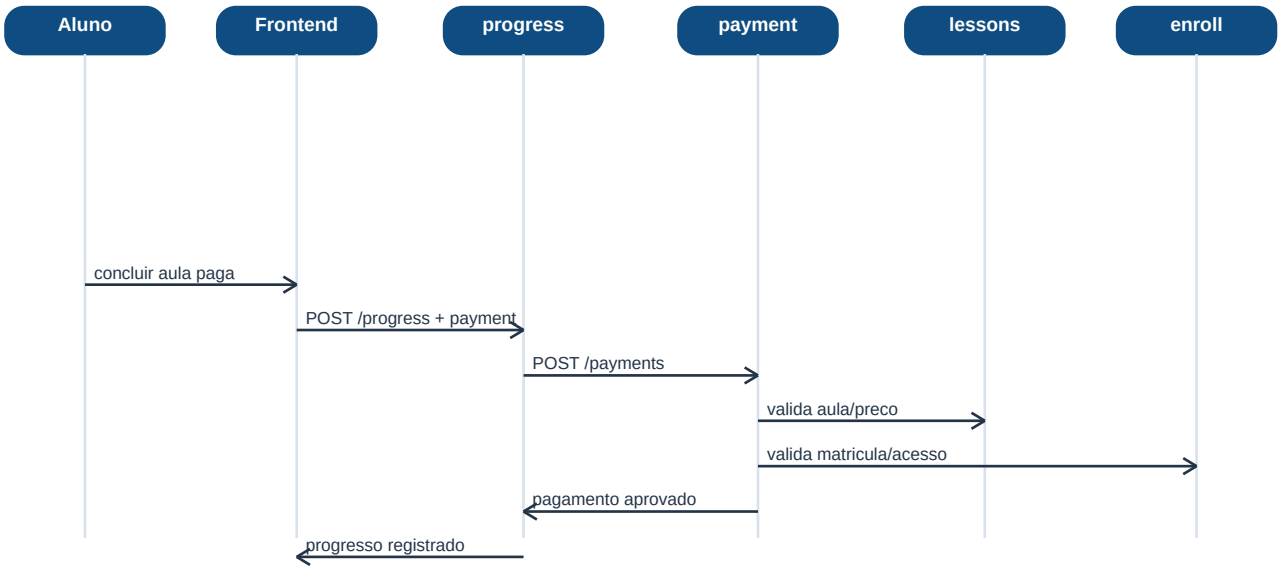
4. Modelo de dados



Relacoes sao logicas: cada microservico possui banco isolado e nao ha foreign keys entre bancos.

Tabela	Campos principais
users	id, name, email, hashed_password, role, created_at
students	auth_user_id, cpf, name, email, whatsapp, telegram, city, state, education_level
courses	title, description, cohort_name, start_date, end_date, capacity, price, is_paid, is_active
enrollments	user_id, student_id, course_id, cpf, group_number, deadlines
lessons	course_id, title, type, order_index, release_week, duration_minutes, price, cards_json
progress_entries	user_id, course_id, lesson_id, completed_at
payments	user_id, course_id, lesson_id, amount, status, provider, card_brand, card_last_four

5. Fluxo de pagamento e progresso



A conclusao de aula paga nasce no frontend, passa pelo progress-service, aciona o payment-service quando necessario e so entao grava o progresso. Essa ordem evita concluir uma aula paga sem pagamento aprovado.

6. Regras de negocio

Regra	Status
Login obrigatorio para operacoes de negocio	Implementado
Roles admin e aluno	Implementado
Perfil por CPF e CPF unico	Implementado
Escolaridade no cadastro do aluno	Implementado
Capacidade maxima de 200 alunos	Implementado
Grupos automaticos de ate 5 alunos	Implementado
Matricula entre 5 e 1 semanas antes do curso	Implementado
Entrega final em 180 dias	Implementado
Acesso por 360 dias	Implementado
Curso com ate 40 aulas	Implementado
Video 30 min, texto 30 min, atividade 60 min	Implementado
Liberacao semanal e ordem sequencial	Implementado
Cards multimidia	Implementado
Pagamento por aula paga na conclusao	Implementado
Progresso individual por aluno/turma	Implementado
Atividade ENADE com 30 questoes objetivas	Pendente
Debito financeiro no final da semana	Pendente
Normalizacao cidade/estado em tabelas proprias	Pendente

7. Frontend

Rota	Descricao
/login	Login e registro
/dashboard	Perfil, matriculas e resumo
/courses	Catalogo e matricula
/lessons	Cards, liberacao, conclusao e pagamento
/admin	Studio Admin para cursos e aulas
/progress	Progresso do aluno ou turma para admin

- Usa VITE_API_URL com default http://localhost:8080.

- Armazena JWT em localStorage como ead-token.

- Exige CPF antes da matricula.

- Abre modal de cartao somente na conclusao de aula paga.

- Admin cria cursos e aulas no Studio Admin.

8. Execucao e validacao

Comando principal:

```
docker compose up -d --build
```

Acessos: frontend http://localhost:3000, gateway http://localhost:8080/health, Swagger dos servicos nas portas 8001 a 8007.

Validacoes realizadas: py_compile dos main.py, import de user-service e lesson-service, build do frontend e docker compose config --quiet.

9. Proximos passos recomendados

- Criar modulo de atividade/prova ENADE com 30 questoes objetivas.
- Especificar e implementar debito no final da semana.
- Normalizar cidades e estados se esse requisito for obrigatorio.
- Adicionar testes automatizados para matricula, liberacao semanal, pagamento e progresso.
- Substituir migracoes em startup por migrations versionadas.